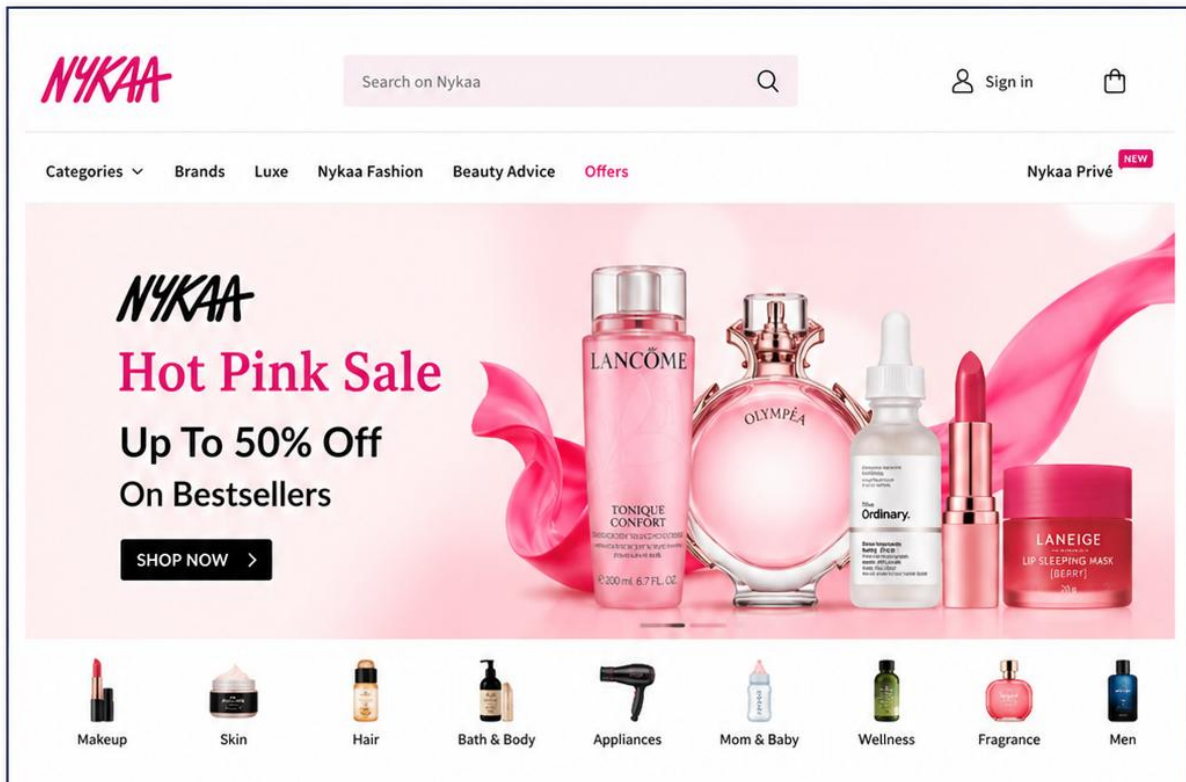




NYKAA E-COMMERCE WEBSITE

Capstone Project and Test Summary

G Hrithicka

Wipro SDET Playwright Training

Project: End-to-End Automation Testing of
Nykaa E-Commerce Website using Playwright and JavaScript

Overview

The Application Under Test (AUT) is the Nykaa E-Commerce website, one of India's leading online beauty and personal care platforms. The website provides users with the ability to browse products, search for items, manage shopping carts and wishlist, view product details, manage user accounts, and proceed through checkout workflows.

The objective of this capstone project was to design and implement an automated testing framework using Playwright with JavaScript. The project focuses on validating major user functionalities, responsive behavior, authentication setup, cross-browser compatibility, and automated reporting.

Goals

1. Business Objectives

- Verify that core user workflows function correctly.
- Ensure seamless navigation across product categories.
- Validate search and product discovery functionality.
- Verify shopping cart and wishlist management.
- Ensure consistent user experience across multiple browsers.
- Validate responsive behaviour on desktop, tablet, and mobile devices.

2. Automation Testing Objectives

- Develop an end-to-end automation framework using Playwright.
- Execute test cases across Chromium, Firefox, and WebKit browsers.
- Implement authentication setup using Playwright storage state.
- Generate execution reports using HTML and Allure reporting.
- Improve test reliability through assertions, hooks, and reusable automation practices.

Specifications

In-Scope Functional Modules

Testing was performed on the following core functional modules:

1. Homepage Validation

- Validate homepage loading, navigation menu visibility, promotional banners, category sections, product images, and homepage accessibility.

2. Product Search

- Validate search bar functionality, product search operations, search result loading, search result visibility, sorting options, and filter visibility.

3. Brand & Category Navigation

- Validate category page navigation, brand navigation, category visibility, product listing visibility, and URL verification.

4. Product Details

- Validate product title visibility, product image visibility, product price visibility, wishlist visibility, discount visibility, and product information display.

5. Cart Management

- Validate cart page accessibility, cart page loading, cart page refresh functionality, and guest-user cart workflows.

6. Wishlist Management

- Validate wishlist page accessibility, wishlist page loading, wishlist navigation, and guest-user wishlist workflows.

7. Checkout Validation

- Validate checkout page accessibility, checkout page loading, page refresh behaviour, and order summary visibility.

8. User Account & Profile Management

- Validate account page accessibility, login navigation, profile page visibility, and guest-user account workflows.

9. Product Recommendation & Discovery

- Validate recommended products, related products, trending products, and product discovery workflows.

10. Responsive UI Validation

- Validate application behaviour across desktop, tablet, and mobile viewports.

Milestone 1 – Analysis & Planning

1.1 Introduction

The objective of this phase is to analyze the functional requirements of the Nykaa E-Commerce website and define an effective automation testing strategy. This phase ensures that all major business functionalities are identified, prioritized, and mapped to appropriate test scenarios before automation implementation.

1.2 Requirement Analysis

The following Functional Requirements (FR) and Non-Functional Requirements (NFR) were identified based on the features available on the Nykaa E-Commerce platform and the scope of automation testing selected for this project.

1.2.1 Functional Requirements (FR)

Module	Requirement Description	Priority
Homepage Validation	Users should be able to access the homepage and view navigation menus, promotional banners, category sections, and product displays.	High
Product Search	Users should be able to search for products and view relevant search results.	High
Brand & Category Navigation	Users should be able to browse products through category and brand navigation options.	High
Product Details	Users should be able to view product information including pricing, images, ratings, and descriptions.	High
Cart Management	Users should be able to access the cart page and view cart-related information.	High
Wishlist Management	Users should be able to access and navigate wishlist-related pages.	Medium

Checkout Validation	Users should be able to access checkout-related pages and validate checkout workflows.	Critical
User Account & Profile Management	Users should be able to access account-related pages and profile-related information.	Medium
Responsive UI Validation	The application should remain accessible and usable across desktop, tablet, and mobile viewports.	High

1.2.2 Non-Functional Requirements (NFR)

- **Browser Compatibility:** The application should function correctly across Chromium, Firefox, and WebKit browsers.
- **Responsive Design:** The application should provide a consistent user experience across desktop, tablet, and mobile screen sizes.
- **Reliability:** Test cases should execute consistently without unexpected failures across supported browsers.
- **Reporting:** Test execution results should be captured using Playwright HTML Reports and Allure Reports.
- **Maintainability:** Test scripts should be organized into modules to support easy maintenance and scalability.
- **Automation Coverage:** Major user-facing functionalities should be automated to improve regression testing efficiency.

1.3 Test Strategy

The testing strategy for the Nykaa E-Commerce Website focuses on validating critical user-facing functionalities through automated testing using Playwright with JavaScript. The strategy is designed to ensure application stability, browser compatibility, and consistent user experience across supported platforms.

The automation approach includes:

- **Cross Browser Testing:** Execution of test cases across Chromium, Firefox, and WebKit browsers to verify browser compatibility.
- **Responsive Testing:** Validation of application behaviour across desktop, tablet, and mobile screen sizes.
- **Authentication Setup:** Implementation of Playwright storage state concepts to manage session-related testing.
- **Automated Reporting:** Generation of Playwright HTML Reports and Allure Reports for execution analysis and result tracking.

The overall strategy aims to maximize automation coverage while maintaining test reliability, readability, and maintainability.

1.4 Requirement Traceability Matrix (RTM)

To ensure complete test coverage, each functional requirement is mapped to one or more corresponding test scenarios.

Requirement	Test Scenario
Homepage Validation	Verify homepage loading, banners, navigation menus, and product visibility.
Product Search	Verify search functionality, search results, sorting, and filters.
Brand & Category Navigation	Verify category navigation, brand navigation, and URL validation.
Product Recommendation & Discovery	Verify recommended products, related products, trending products, and product discovery workflows.
Product Details	Verify product title, price, images, discounts, and product information.
Cart Management	Verify cart page accessibility, page refresh, and guest-user workflows.
Wishlist Management	Verify wishlist page accessibility and navigation workflows.

Checkout Validation	Verify checkout page loading, validation workflows, and order summary visibility.
User Account & Profile Management	Verify account page accessibility and profile-related navigation.
Responsive UI Validation	Verify application behaviour across desktop, tablet, and mobile viewports.

Milestone 2 – Static Review and Manual Test Design

2.1 Static Review of Automation Framework

Static testing was performed by reviewing the project structure, configuration files, and automation framework setup without executing the test cases. The purpose of static review was to ensure that the Playwright project was organized, maintainable, and suitable for automation execution.

The following areas were reviewed:

- Module-wise folder structure under the tests directory.
- Playwright configuration file for browser projects, reporters, retries, workers, timeout, and execution settings.
- Proper usage of .gitignore to exclude generated reports, node_modules, test-results, allure-results, allure-report, and auth.json.
- Verification of test file naming conventions using .spec.js files.
- Review of authentication setup files such as auth.setup.js and auth.json.
- Verification that test cases are grouped module-wise using test.describe().
- Review of assertions, hooks, test steps, soft assertions, polling assertions, and responsive validation logic.



Figure 2.1: Playwright Framework Structure

2.2 Manual Exploratory Testing

Before automating the test cases, basic manual exploratory testing was performed on the Nykaa website to understand the application flow and identify stable test scenarios. The website was manually checked for homepage loading, product search, category navigation, product details, cart page behaviour, wishlist page behaviour, checkout page accessibility, account page navigation, and responsive behaviour.

Manual exploration helped in identifying which flows could be automated reliably and which flows were unstable due to OTP restrictions or dynamic website behaviour. Since Nykaa OTP login did not persist successfully inside the automation browser, authenticated flows were not forced into the main test execution. Instead, guest-user workflows were validated for stability.

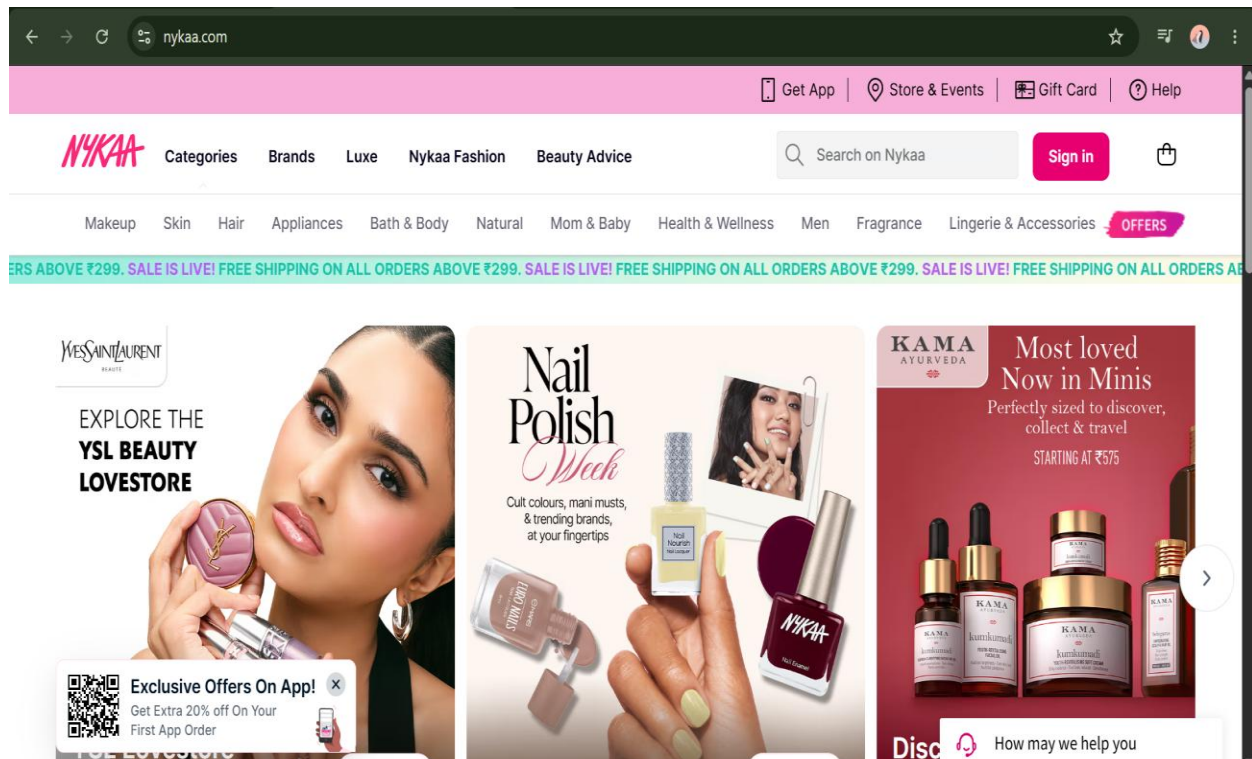


Figure 2.2: Nykaa Homepage Used for Manual Exploration

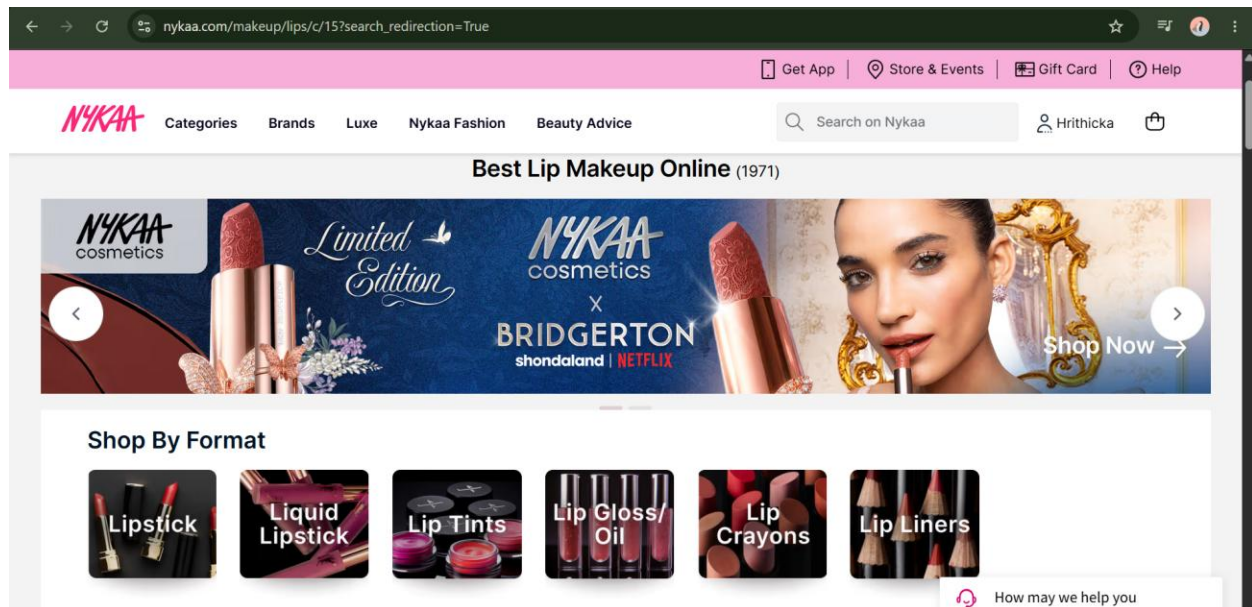


Figure 2.3: Product Search Results Page

Manual exploration helped in identifying which flows could be automated reliably and which flows were unstable due to OTP restrictions or dynamic website behaviour. Since Nykaa OTP login did not persist successfully inside the automation browser, authenticated flows were not forced into the main test execution. Instead, guest-user workflows were validated for stability.

2.3 Test Scenario Design

Based on the manual exploration and requirement analysis, test scenarios were designed for each module.

The scenarios focused on page accessibility, UI visibility, navigation validation, search functionality, product information display, cart and wishlist page behaviour, checkout validation, account page behaviour, and responsive UI validation.

The test cases were designed to be executed across Chromium, Firefox, and WebKit browsers using Playwright.

Stable assertions were selected to reduce failures caused by dynamic content changes on the live Nykaa website.

```

JS homepage.spec.js X
tests > homepage_testing > JS homepage.spec.js > test.describe('Homepage Validation Module') callback >
1  const { test, expect } = require('@playwright/test');
2
3  test.describe('Homepage Validation Module', () => {
4
5      test.beforeEach(async ({ page }) => {
6
7          await page.goto('https://www.nykaa.com', {
8              waitUntil: 'domcontentloaded',
9              timeout: 60000
10         });
11     });
12
13     // Test 1 - Verify homepage title
14
15     test('Verify homepage title', async ({ page }) => {
16
17         await expect(page).toHaveTitle(/Nykaa/i);
18     });
19
20     // Test 2 - Verify promotional banner visibility
21
22     test('Verify promotional banner visibility', async ({ page }) => {
23
24         await expect(page.locator('body')).toContainText(/OFF|Offer/i);
25     });
26
27
28

```

Figure 2.4: Homepage Module Test Scenarios

```

/* Configure projects for major browsers */
projects: [
    {
        name: 'chromium',
        use: {
            ...devices['Desktop Chrome'],
            channel: 'chrome'
        },
    },
    {
        name: 'firefox',
        use: { ...devices['Desktop Firefox'] },
    },
    {
        name: 'webkit',
        use: { ...devices['Desktop Safari'] },
    },
]

```

Figure 2.5: Cross Browser Configuration in Playwright

Milestone 3 – Playwright Automation Testing

3.1 Automation Framework Setup

The automation framework for the Nykaa E-Commerce Website was developed using Playwright with JavaScript. The framework was designed to support scalable test execution, module-wise organization, cross-browser testing, responsive validation, and automated reporting.

The following technologies and tools were used during framework development:

- **Playwright:** Automation testing framework used for browser automation and test execution.
- **JavaScript:** Programming language used for writing automation scripts.
- **Node.js:** Runtime environment used for Playwright execution and package management.
- **Visual Studio Code:** Development environment used for script creation and maintenance.
- **GitHub:** Version control platform used for source code management and project tracking.
- **Allure Report:** Reporting framework used to generate detailed execution reports.
- **Playwright HTML Report:** Built-in reporting mechanism used to review execution results and test outcomes.



```
reporter: [['html'], ['allure-playwright', { resultsDir: 'allure-results' }]],
/* Shared settings for all the projects below. See https://playwright.dev/docs/api/cli
use: {
  /* Base URL to use in actions like `await page.goto(')`. */
  // baseURL: 'http://localhost:3000',

  /* Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-viewer
  headless: process.env.CI ? true : false,
  trace: 'on-first-retry',
},

/* Configure projects for major browsers */
projects: [
  {
    name: 'chromium',
    use: {
      ...devices['Desktop Chrome'],
      channel: 'chrome'
    },
  },

  {
    name: 'firefox',
    use: { ...devices['Desktop Firefox'] },
  },

  {
    name: 'webkit',
    use: { ...devices['Desktop Safari'] },
  },
],
```

Figure 3.1: Playwright Framework Configuration

The framework was configured to execute test cases across Chromium, Firefox, and WebKit browsers. Module-wise test organization was implemented to improve maintainability and support efficient execution of test suites.

3.2 Automation Features Implemented

The Playwright automation framework was developed using several built-in features to improve test reliability, maintainability, and execution efficiency.

The following automation features were implemented during the project:

- **Assertions:** Assertions were used to validate page visibility, URL correctness, content verification, and application behaviour.
- **Soft Assertions:** Soft assertions were implemented to allow multiple validations within a single test execution without immediately terminating the test on the first failure.
- **Polling Assertions:** Polling assertions were used to repeatedly verify application states until the expected condition was satisfied.
- **Hooks:** Playwright hooks such as `beforeEach()` were used to perform common setup activities before test execution.
- **test.step():** Test steps were implemented to improve execution readability and provide better reporting visibility.
- **Cross Browser Testing:** Test cases were executed across Chromium, Firefox, and WebKit browsers to validate browser compatibility.
- **Responsive Testing:** Validation was performed across desktop, tablet, and mobile viewports to ensure responsive behaviour.
- **Authentication Setup:** Playwright storage state concepts were implemented using `auth.setup.js` and `auth.json` for authentication-related testing.

These features improved the overall quality of the automation framework and enabled efficient execution of test scenarios across multiple modules.

```

// Test 11 - Verify responsive layout using test steps

test('Verify responsive layout using test steps', async ({ browser }) => {
  await test.step('Check mobile viewport', async () => {
    const mobileContext = await browser.newContext({
      viewport: { width: 375, height: 667 }
    });

    const mobilePage = await mobileContext.newPage();

    await mobilePage.goto(homeUrl, {
      waitUntil: 'domcontentloaded',
      timeout: 60000
    });
  });
});

```

Figure 3.2: Implementation of Test Steps and Assertions in Playwright

3.3 Module Coverage

The automation framework was divided into multiple modules to improve maintainability, execution efficiency, and test coverage. Each module was designed to validate a specific area of the Nykaa E-Commerce application.

Module Name	Coverage Area
Homepage Validation Module	Homepage loading, navigation menus, banners, featured sections.
Product Search Module	Search functionality, search results, sorting, filters.
Brand & Category Navigation Module	Brand navigation, category traversal, URL validation.
Product Recommendation & Discovery Module	Recommended products, related products, trending products.
Product Details Module	Product title, pricing, images, discounts, product information.

Cart Management Module	Cart page accessibility, refresh validation, guest-user workflows.
Wishlist Management Module	Wishlist accessibility, navigation, guest-user validation.
Checkout Validation Module	Checkout page validation, order summary visibility, page behaviour.
User Account & Profile Management Module	Account page accessibility, login navigation, profile workflows.
Responsive UI Validation Module	Desktop, tablet, and mobile viewport validation

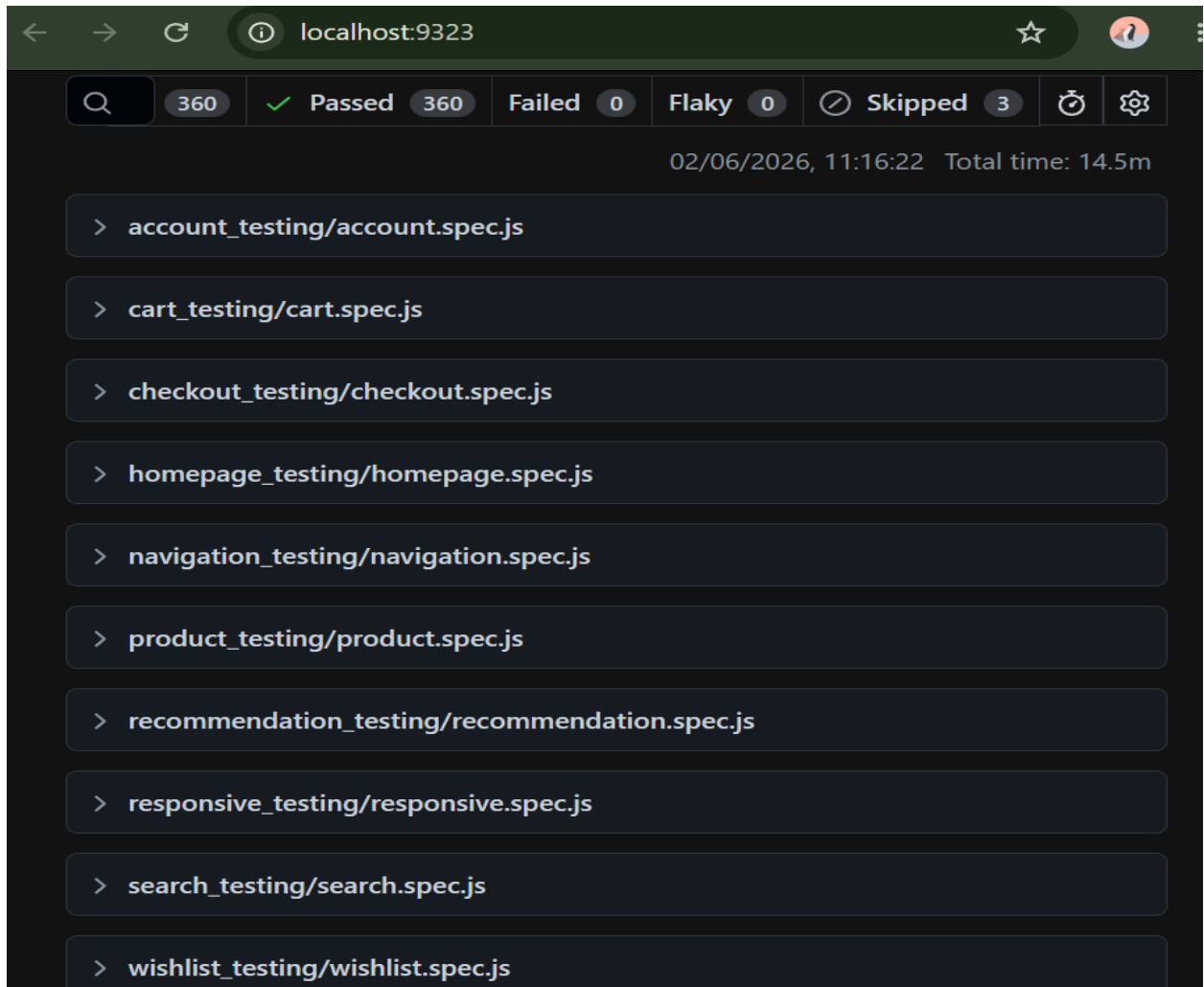


Figure 3.3: Module-Wise Test Coverage

3.4 Authentication Setup

Authentication setup was implemented using Playwright's storage state functionality to understand session management and authentication concepts within the automation framework.

An authentication setup file named `auth.setup.js` was created to generate a storage state file (`auth.json`). The storage state file stores browser session information, cookies, and local storage data, allowing session reuse during test execution.

The authentication setup was implemented as part of the Playwright framework to demonstrate storage state management and session handling concepts. The generated `auth.json` file was used to understand how authenticated browser states can be captured and reused within automation projects.

The following components were implemented as part of the authentication setup:

- **auth.setup.js:** Used to create and save the browser storage state.
- **auth.json:** Stores session-related information generated during authentication setup.

This implementation demonstrates Playwright authentication concepts and storage state management within the automation framework.

```
22 process.stdin.once('data', async () => {
23
24     await context.storageState({
25         path: 'auth.json'
26     });
27
28     await browser.close();
29
30     console.log('auth.json created successfully');
31
32 });
33
34 })();
```

Figure 3.4: Authentication Setup Using Playwright Storage State

3.5 Test Execution Summary

The automated test suite was executed across multiple browsers using Playwright. The execution covered all identified modules and validated major user-facing functionalities of the Nykaa E-Commerce website.

Execution was performed across Chromium, Firefox, and WebKit browsers to ensure browser compatibility and consistent application behaviour.

Final Execution Results:

Metric	Result
Total Modules	10
Total Executed Test Cases	363
Passed	360
Skipped	3
Failed	0

Browser-Wise Execution Summary:

Browser	Executed Test Cases
Chromium	120
Firefox	120
WebKit	120

The execution results demonstrate successful automation coverage across all implemented modules. The generated reports provide detailed execution statistics, module-wise coverage information, and browser-wise execution results.

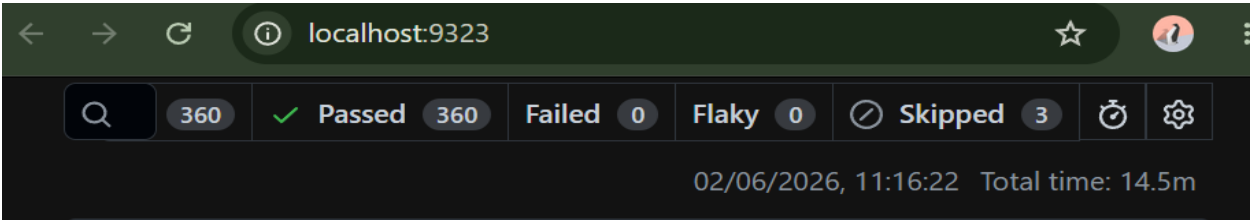


Figure 3.5: Final Playwright Test Execution Summary

Milestone 4 – Allure Reports

4.1 Allure Report Generation

Allure Report was implemented to generate detailed and interactive test execution reports for the Nykaa E-Commerce automation project. The Allure report provides clear visibility of test execution status, module-wise results, browser-wise execution, execution duration, passed test cases, skipped test cases, and failed test cases.

The report was generated after executing the complete Playwright test suite across Chromium, Firefox, and WebKit browsers.

Final Allure Report Summary:

Metric	Result
Total Test Cases	363
Passed	360
Skipped	3
Failed	0
Success Rate	100%
Browsers Covered	Chromium, Firefox, WebKit

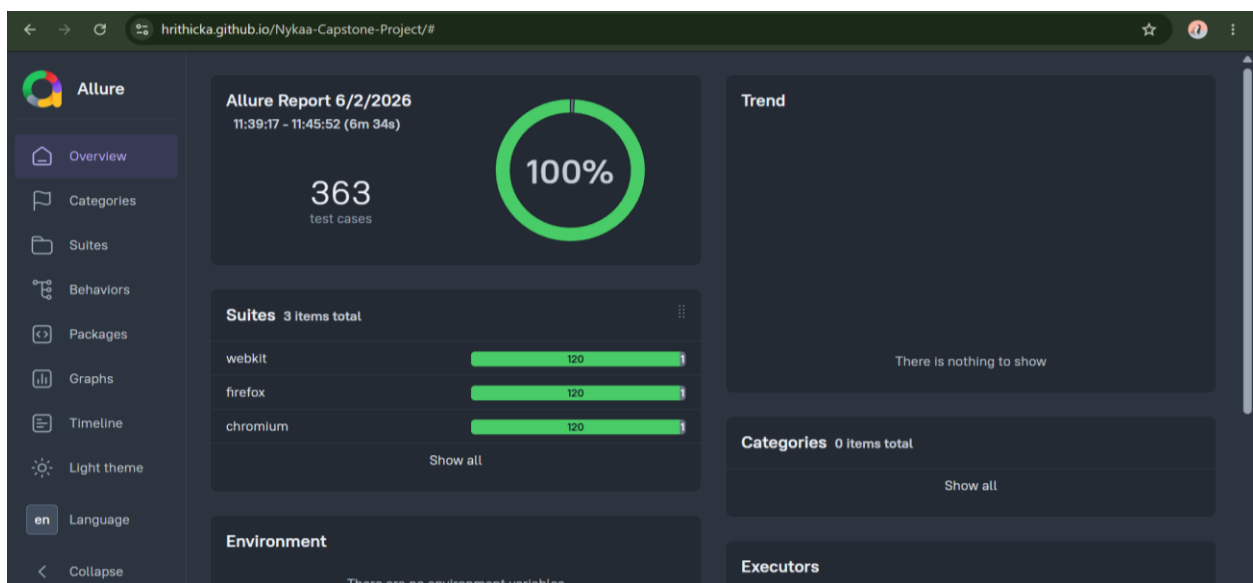


Figure 4.1: Allure Report Overview Dashboard

The Allure report helped in analyzing test execution results in a structured way and provided better visibility compared to terminal output.

4.2 GitHub Repository Structure

The complete automation framework was maintained in a GitHub repository to support version control, project organization, and collaboration. The repository contains module-wise test files, configuration files, authentication setup files, workflow definitions, and reporting configurations. GitHub was also used to integrate GitHub Actions for automated workflow execution.

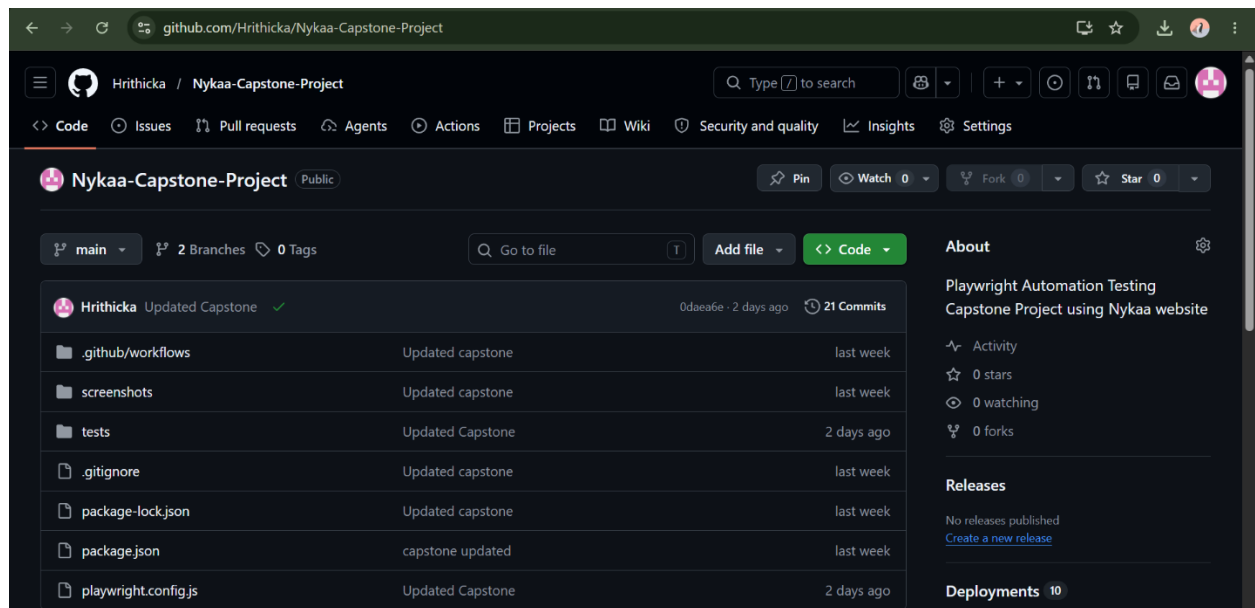


Figure 4.2: GitHub Repository Structure

4.3 GitHub Actions Workflow

GitHub Actions was implemented to automate the execution of Playwright test cases through a Continuous Integration (CI) workflow. The workflow was configured using a YAML file within the .github/workflows directory. Whenever code changes were pushed to the GitHub repository, the workflow automatically executed the Playwright test suite and generated execution results.

The implementation of GitHub Actions helped automate test execution, improve consistency, and demonstrate CI/CD concepts within the automation framework.

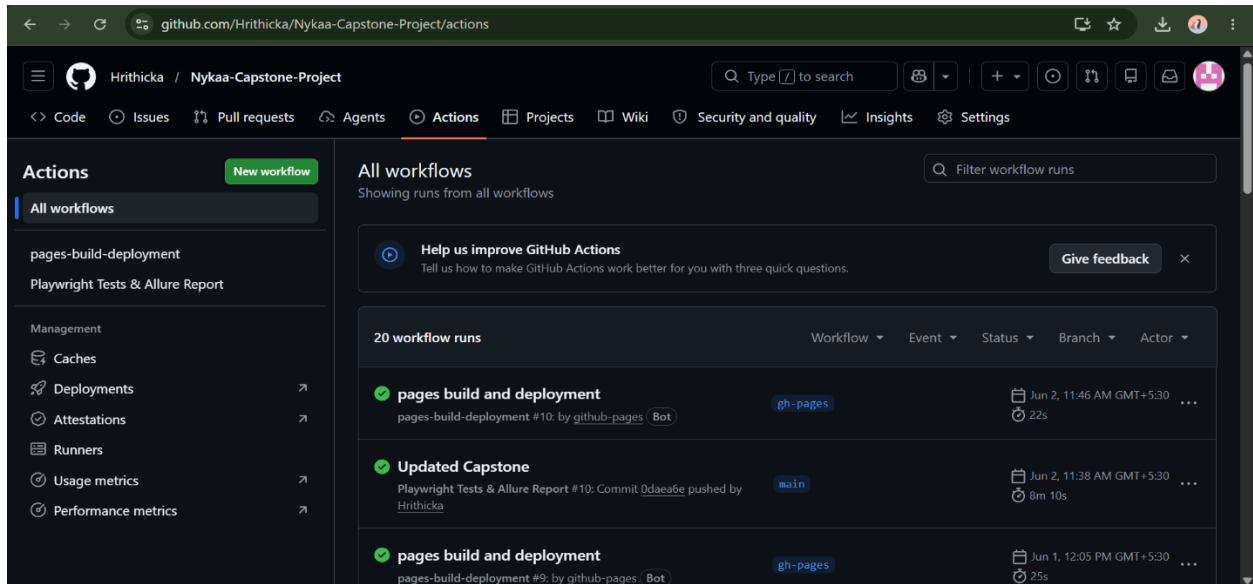


Figure 4.3: GitHub Actions Workflow Execution

4.4 Allure Graphical Analysis

The Allure Report provides graphical visualization of test execution results. The graphs help analyze browser-wise execution, test distribution, execution trends, and overall project outcomes. These visual reports improve result interpretation and provide a better understanding of framework execution statistics.

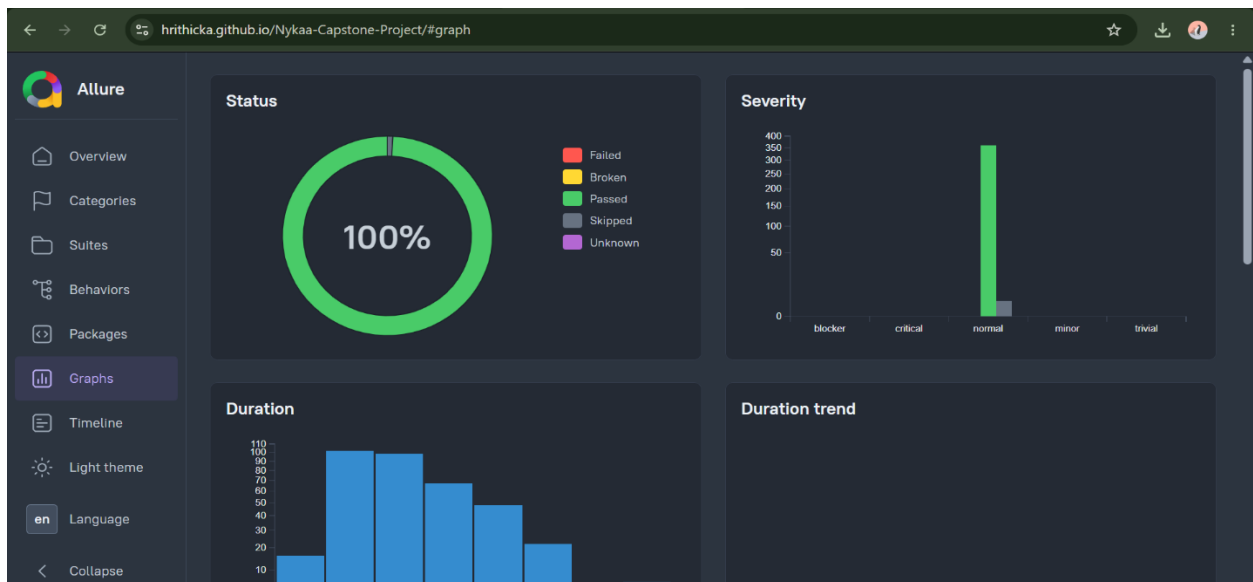


Figure 4.4: Allure Graphical Analysis Dashboard

Conclusion

The Nykaa E-Commerce Website Automation Testing Project was successfully developed using Playwright with JavaScript. The project automated major functionalities including homepage validation, product search, category navigation, product details, cart management, wishlist management, checkout validation, user account management, product recommendation, and responsive UI validation.

The automation framework was executed across Chromium, Firefox, and WebKit browsers and generated detailed execution reports using Playwright HTML Reports and Allure Reports.

The final execution achieved 363 total test cases with 360 passed and 3 skipped test cases, demonstrating successful automation coverage and framework reliability.

Overall, the project provided practical experience in Playwright automation, cross-browser testing, responsive testing, authentication concepts, and automated reporting.